

Subway Transit system

Project Summery

Our project is a subway transit management database designed to manage and give data about subway trains, passengers, transit passes, routes, stations, gates, and fare zones. The system stores vehicle and schedule information, verifies whether passenger transit passes are valid, and records passenger entry and exit activity at stations.

Schema Changes

A total of 4 tables were removed, 2 entities in an ISA and 2 on relationships. The new schema still fits the requirement milestone 2, as shown in the previous ER diagram.

1. Removed Transport Table

The revised database focuses specifically on the subway system, so a generic transportation superclass was no longer necessary.

The attributes that were previously stored in `Transport` were moved directly into `SubwayTrain` :

```
ManufactureYear NUMBER(4) NOT NULL,  
Capacity NUMBER(4) NOT NULL,  
WheelchairAccessibility CHAR(1) NOT NULL
```

The foreign key from `SubwayTrain` to `Transport` was also removed.

2. Removed Bus Table

The entire `Bus` entity was removed.

3. Modified SubwayTrain

The `SubwayTrain` table now contains the vehicle information that was previously stored in `Transport` .

4. Changed ID Data Types

Many identifiers were changed from fixed-length `CHAR(10)` to variable-length `VARCHAR2(10)` .
`VARCHAR2` avoids unnecessary space padding associated with `CHAR` .

5. Added Default Passenger Category

The `PassengerCategory` attribute now has a default value.

If a passenger is inserted without specifying a category, the database automatically assigns it as Adult.

6. Simplified Pass Status

The allowed travelling statuses were reduced to Active and Inactive.

7. Added `RouteNumber`

A new `RouteNumber` attribute was added to the `Route` table.

A route now has a unique route number that can be used to identify it in addition to the internal `RouteID` .

8. Removed `Direction` from `Route`

`Direction` is no longer stored as an attribute of `Route` in the revised schema.

9. Changed Schedule Time Representation

The schedule time attributes were changed from `TIMESTAMP` to `VARCHAR2(5)` .

The values are intended to use the `HH:MM` format, for example:

```
08:30
14:45
23:00
```

Constraints were added to ensure valid times:

10. Changed `RunsOn` Foreign Key

Before

`RunsOn.VehicleID` referenced the general `Transport` table:

Since `Transport` was removed, a vehicle running on a route must now be a subway train.

11. Removed `ValidEntranceAt`

The `ValidEntranceAt` relationship table was removed since pass entrance can be validated by using the pass table and the passEntrance table.

12. Removed `ValidExitAt`

The `ValidExitAt` relationship table was also removed.

Populated Database Tables

This document shows the populated contents of all 15 database tables after `baseData.sql` is ran

1. TrainModel

ModelName	Capacity
Mark I	4
Mark II	2
Mark III	4
Mark V	6
Alstom	5

2. SubwayTrain

VehicleID	TrainSetNumber	ModelName	ManufactureYear	Capacity	WheelchairAccessibility
V000000001	TS-100	Mark I	2015	60	Y
V000000002	TS-101	Mark II	2018	45	Y
V000000003	TS-102	Mark III	2010	80	N
V000000004	TS-103	Mark V	2021	120	Y
V000000005	TS-104	Alstom	2020	100	Y

3. Passenger

PassengerID	FirstName	LastName	PassengerCategory
P000000001	John	Doe	Adult
P000000005	Charlie	Davis	Adult
P000000006	David	Wilson	Adult
P000000002	Jane	Smith	Student
P000000007	Emma	Taylor	Student
P000000008	Liam	Anderson	Student
P000000003	Alice	Johnson	Senior
P000000009	Robert	Martin	Senior
P000000010	Susan	Thompson	Senior
P000000004	Bob	Brown	Child
P000000011	Emily	White	Child

4. Passes

TicketID	PassengerID	AmountPaid	TravellingStatus	PurchaseTime
T000000001	P000000001	105.00	Active	CURRENT_TIMESTAMP
T000000002	P000000002	55.00	Active	CURRENT_TIMESTAMP
T000000003	P000000002	100.00	Active	CURRENT_TIMESTAMP
T000000004	P000000003	45.00	Active	CURRENT_TIMESTAMP
T000000005	P000000003	30.00	Active	CURRENT_TIMESTAMP
T000000006	P000000003	105.00	Active	CURRENT_TIMESTAMP
T000000007	P000000004	30.00	Active	CURRENT_TIMESTAMP
T000000008	P000000005	55.00	Active	CURRENT_TIMESTAMP
T000000009	P000000005	100.00	Active	CURRENT_TIMESTAMP
T000000010	P000000006	55.00	Active	CURRENT_TIMESTAMP
T000000011	P000000007	30.00	Active	CURRENT_TIMESTAMP

T000000012	P000000008	105.00	Active	CURRENT_TIMESTAMP
T000000013	P000000009	55.00	Active	CURRENT_TIMESTAMP
T000000014	P000000010	45.00	Active	CURRENT_TIMESTAMP
T000000015	P000000003	105.00	Active	CURRENT_TIMESTAMP
T000000016	P000000003	105.00	Active	CURRENT_TIMESTAMP

5. ZonePass

TicketID	ZoneID
T000000001	3
T000000002	1
T000000004	2
T000000005	1
T000000007	2
T000000009	3
T000000010	1
T000000012	3
T000000014	2

6. TimedPass

TicketID	StartTime	EndTime	PassType
T000000003	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 7 days	Weekly
T000000006	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 1 month	Monthly
T000000008	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 1 day	Daily
T000000011	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 1 day	Daily
T000000013	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 7 days	Weekly
T000000015	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 1 days	Daily
T000000016	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP + 7 days	Weekly

7. Route

RouteID	RouteNumber	RouteName	RouteDescription	Colour
R000000001	99	99 B-Line	Express to UBC	Orange
R000000002	1	Expo Line	Downtown to Surrey	Blue
R000000003	2	Canada Line	Richmond to Downtown	Light Blue
R000000004	3	Millennium	VCC to Lafarge	Yellow
R000000005	49	49	Metrotown to UBC	Red

8. Schedule

RouteID	StartTime	DayType	EndTime	Frequency
R000000001	06:00	Weekday	23:00	5
R000000001	07:00	Weekend	22:00	10
R000000001	08:00	Holiday	21:00	15
R000000002	05:00	Weekday	01:00	3
R000000002	06:00	Weekend	01:00	5
R000000002	07:00	Holiday	00:00	8
R000000003	05:30	Weekday	00:30	5
R000000003	06:00	Weekend	00:30	7
R000000003	07:00	Holiday	23:30	10
R000000004	05:30	Weekday	00:30	4
R000000004	06:30	Weekend	00:30	7
R000000004	07:00	Holiday	23:00	10
R000000005	05:00	Weekday	23:30	10
R000000005	07:00	Weekend	23:00	15
R000000005	08:00	Holiday	22:00	20

9. Zone

ZoneID	ZoneName	Colour	Description
--------	----------	--------	-------------

1	Vancouver	Green	Central Hub
2	Burnaby/Richmond	Yellow	Inner Suburbs
3	Surrey/Langley	Red	Outer Suburbs
4	North Shore	Blue	Mountains
5	Delta	Orange	South of Fraser

10. TransportStation

StationID	ZoneID	StationName	Address	Accessibility
S000000001	1	Waterfront	601 W Cordova St	Y
S000000002	2	Metrotown	6300 Central Blvd	Y
S000000003	3	Surrey Central	10275 City Pkwy	N
S000000004	1	UBC Exchange	6131 Student Union Blvd	Y
S000000005	2	Richmond-Brighouse	6971 No 3 Rd	Y
S000000006	1	Commercial-Broadway	1701 E Broadway	Y
S000000007	1	Joyce-Collingwood	5094 Joyce St	Y
S000000008	1	Marine Drive	8575 Cambie St	Y
S000000009	1	VCC-Clark	115 W 1st St	Y
S000000010	2	Lougheed Town Centre	9855 Austin Rd	Y
S000000011	1	Langara-49th	535 W 49th Ave	Y

11. RouteStop

RouteID	StopNumber	StationID
R000000001	1	S000000004
R000000001	2	S000000006
R000000001	3	S000000007
R000000002	1	S000000001
R000000002	2	S000000006
R000000002	3	S000000003

R000000003	1	S000000005
R000000003	2	S000000008
R000000003	3	S000000001
R000000004	1	S000000009
R000000004	2	S000000006
R000000004	3	S000000010
R000000005	1	S000000002
R000000005	2	S000000011
R000000005	3	S000000004

12. Gate

GateID	StationID	Status	EnterOrExit	TotalEnterExitCount
G000000001	S000000001	Active	Enter	15
G000000002	S000000001	Active	Exit	145
G000000003	S000000001	Inactive	Enter	50
G000000004	S000000002	Active	Enter	80
G000000005	S000000002	Active	Exit	78
G000000006	S000000002	Inactive	Exit	32
G000000007	S000000003	Active	Enter	120
G000000008	S000000003	Active	Exit	115
G000000009	S000000003	Inactive	Enter	45
G000000010	S000000004	Active	Enter	500
G000000011	S000000004	Active	Exit	480
G000000012	S000000004	Inactive	Exit	180
G000000013	S000000005	Active	Enter	65
G000000014	S000000005	Active	Exit	62
G000000015	S000000005	Inactive	Enter	21
G000000016	S000000006	Active	Enter	110

G000000017	S000000006	Active	Exit	105
G000000018	S000000006	Inactive	Exit	40
G000000019	S000000007	Active	Enter	70
G000000020	S000000007	Active	Exit	68
G000000021	S000000007	Inactive	Enter	25
G000000022	S000000008	Active	Enter	45
G000000023	S000000008	Active	Exit	43
G000000024	S000000008	Inactive	Exit	15
G000000025	S000000009	Active	Enter	35
G000000026	S000000009	Active	Exit	33
G000000027	S000000009	Inactive	Enter	90
G000000028	S000000010	Active	Enter	60
G000000029	S000000010	Active	Exit	58
G000000030	S000000010	Inactive	Exit	19
G000000031	S000000011	Active	Enter	40
G000000032	S000000011	Active	Exit	38
G000000033	S000000011	Inactive	Enter	12

13. RunsOn

VehicleID	RouteID
V000000001	R000000001
V000000002	R000000002
V000000003	R000000003
V000000004	R000000004
V000000005	R000000005

14. PassEnter

TicketID	GateID	EntryTime
----------	--------	-----------

T000000001	G000000001	CURRENT_TIMESTAMP
T000000002	G000000003	CURRENT_TIMESTAMP
T000000003	G000000005	CURRENT_TIMESTAMP
T000000004	G000000001	CURRENT_TIMESTAMP
T000000005	G000000003	CURRENT_TIMESTAMP

15. PassExit

TicketID	GateID	ExitTime
T000000001	G000000002	CURRENT_TIMESTAMP
T000000002	G000000004	CURRENT_TIMESTAMP
T000000003	G000000002	CURRENT_TIMESTAMP
T000000004	G000000004	CURRENT_TIMESTAMP
T000000005	G000000002	CURRENT_TIMESTAMP

Project Query Requirements

This document lists the functions that satisfy the project query requirements. Each requirement includes the implemented function and links to its corresponding **function specification** and **API specification**.

| Requirement | Function | Code | Function Specs | API Specs |

|—|—|—|—|

| **Insert** | purchasePassForPassenger() | services/passService.js line: 5 | [Function Specs](#) | [API Specs](#) |

| **Select** | getPassesByPassengerId() | services/passService.js line: 453 | [Function Specs](#) | [API Specs](#) |

| **Update** | updatePassenger() | services/passengerService.js line: 181 | [Function Specs](#) | [API Specs](#) |

| **Join** | enterGate() | services/gateService.js line: 201 | [Function Specs](#) | [API Specs](#) |

| **Delete** | deletePass() | services/passService.js line: 511 | [Function Specs](#) | [API Specs](#) |

| **Projection** | getPassengerById() | services/passengerService.js line: 60 | [Function Specs](#) | [API Specs](#) |

| **GROUP BY** | getAverageSpendingByPassengerCategory() | services/statsService.js line: 4 | [Function Specs](#) | [API Specs](#) |

| **HAVING** | getStationsWithMoreThan130Events() | services/statsService.js line: 35 | [Function Specs](#) | [API Specs](#) |

| **Nested Aggregation with GROUP BY** | getStationsServedByMoreThanAverageRoutes() | services/statsService.js line: 77 | [Function Specs](#) | [API Specs](#) |

| **Division** | getPassengersWithAllPassTypesCount() | services/statsService.js line: 129 | [Function Specs](#) | [API Specs](#) |

Requirement Summary

The project query requirements are satisfied by the following database operations:

1-6:

- **Insert** — Buys a new pass for a passenger using passenger ID
- **Select** — Retrieves all passes belonging to a specific passenger.
- **Update** — Updates information of a passenger
- **Join** — Retrieves information by joining data across related tables when a passenger enters a gate.
- **Delete** — Deletes a specific pass belonging to a passenger.
- **Projection** — Allows the user to select which passenger attributes are returned and in what order.

7-10:

- **GROUP BY** — Calculates average spending grouped by passenger category.

```
SELECT
  p.PassengerCategory,
  COUNT(DISTINCT p.PassengerID) AS PassengerCount,
  AVG(pa.AmountPaid) AS AverageAmountSpent
FROM Passenger p
JOIN Passes pa
  ON p.PassengerID = pa.PassengerID
GROUP BY
  p.PassengerCategory
ORDER BY
  p.PassengerCategory
```

The query groups passengers by PassengerCategory and calculates the number of unique passengers by ID and their average pass spending within each category. The results are then sorted alphabetically by passenger category.

- **HAVING** — Finds stations with more than 130 total entrance/exit events for a given entry/exit type.

```
SELECT
  StationID,
  EnterOrExit,
  SUM(TotalEnterExitCount) AS TotalEnterExitCount
FROM Gate
GROUP BY
  StationID,
  EnterOrExit
HAVING
  SUM(TotalEnterExitCount) > 130
```

```
ORDER BY
    TotalEnterExitCount DESC;
```

The query **aggregates the total number of entrance or exit events for each station**, separately for entrances and exits. It groups the results by station and entry/exit type, calculates the total using `SUM()` , and uses `HAVING` to keep only groups with more than 130 events. The qualifying results are displayed in descending order of total event count.

- **Nested Aggregation with GROUP BY** — Finds stations served by at least the average number of routes.

```
SELECT
    s.StationID,
    s.StationName,
    COUNT(DISTINCT rs.RouteID) AS RouteCount
FROM TransportStation s
LEFT JOIN RouteStop rs
    ON s.StationID = rs.StationID
GROUP BY
    s.StationID,
    s.StationName
HAVING COUNT(DISTINCT rs.RouteID) >= (
    SELECT AVG(route_count)
    FROM (
        SELECT
            s2.StationID,
            COUNT(DISTINCT rs2.RouteID) AS route_count
        FROM TransportStation s2
        LEFT JOIN RouteStop rs2
            ON s2.StationID = rs2.StationID
        GROUP BY s2.StationID
    )
)
ORDER BY
    RouteCount DESC
```

The query finds stations served by at least the average number of distinct routes across all stations. It groups stations by their route count, compares each count to the overall average using a nested aggregation, drops the group of stations if it has a lower route count, and orders the results from highest to lowest route count.

- **Division** — Counts passengers who have purchased every required pass type: Zone, Daily, Weekly, and Monthly.

```
SELECT COUNT(*) AS PassengerCount
FROM Passenger p
WHERE NOT EXISTS (
```

```

SELECT 1
FROM (
    SELECT 'Zone' AS PassType
    FROM dual

    UNION ALL

    SELECT 'Daily' AS PassType
    FROM dual

    UNION ALL

    SELECT 'Weekly' AS PassType
    FROM dual

    UNION ALL

    SELECT 'Monthly' AS PassType
    FROM dual
) requiredTypes
WHERE NOT EXISTS (
    SELECT 1
    FROM Passes pa
    LEFT JOIN ZonePass zp
        ON pa.TicketID = zp.TicketID
    LEFT JOIN TimedPass tp
        ON pa.TicketID = tp.TicketID
    WHERE pa.PassengerID = p.PassengerID
    AND (
        (
            requiredTypes.PassType = 'Zone'
            AND zp.TicketID IS NOT NULL
        )
        OR
        (
            requiredTypes.PassType IN (
                'Daily',
                'Weekly',
                'Monthly'
            )
            AND tp.PassType = requiredTypes.PassType
        )
    )
)
)
)

```

The query counts passengers who have purchased at least one Zone pass, one Daily pass, one Weekly pass, and one Monthly pass. It uses nested NOT EXISTS conditions to implement relational division: for each passenger, the query checks that there is no required pass type that the passenger is missing.

AI Acknowledgements

AI has been used throughout this project to generate data for populating tables, provide suggestions for architectural and code refactoring, assist with code review and debugging, and generate project documentation.

Populating Tables

AI was provided with the revised database schema and asked to generate data for populating each table according to user-defined requirements.

For example, the AI was given requirements such as:

There needs to be at least 2 passengers in each passenger category.

The generated data was then reviewed by the student and modified to ensure that it satisfied the project requirements and database constraints.

Refactoring Suggestions

AI was used to provide both architectural refactoring suggestions and code-level refactoring suggestions.

Architectural Refactoring

The original application structure from the demo project was described to the AI, along with requirements for parallel implementation of modules and backend testing. The AI provided several possible architectural structures.

The student evaluated these suggestions and selected an appropriate structure. The demo project was then refactored and tested by the student according to the selected design.

Code Refactoring, Code Review, and Debugging

For code refactoring, code review, and debugging, the student provided code that they had written to the AI. The AI reviewed the code and identified potential bugs, code quality issues, refactoring opportunities, and naming improvements.

The student then reviewed the suggestions and made appropriate changes to the code.

Generating Documentation

To support group collaboration and improve development efficiency, AI was used to assist in generating project documentation. The documentation generated with AI assistance includes:

- ARCHITECTURE.md
- SETUP.md
- TESTING.md
- Documentation within the APIs folder
- Documentation within the specs folder

Specs Documentation

For documentation in the specs folder, the student provided their implementation code to the AI along with a short description of the purpose and functionality of the code.

The AI generated function specification documentation based on the provided information. The student then reviewed and edited the generated documentation.

API Documentation

For documentation in the APIs folder, the student provided the relevant controller code together with the previously generated specs documentation.

The AI used these materials to generate API documentation describing the endpoints, requests, responses, and functionality. The student then reviewed and edited the generated documentation.

ARCHITECTURE.md and TESTING.md

ARCHITECTURE.md and TESTING.md were initially generated with AI assistance during the architectural refactoring process.

As development progressed, these documents were co-edited by the student and AI to reflect issues encountered while running and testing the program, as well as changes made to the application's architecture and testing procedures.

SETUP.md

SETUP.md was generated with AI assistance based on the setup instructions provided in Tutorial 6. The generated documentation was then reviewed and edited by the student.